

A series of four parallel white diagonal lines of varying lengths, located in the bottom-left corner of the slide.

DAY 2 AGENDA

- ▶ Tags
- ▶ Variables
- ▶ Loops
- ▶ When
- ▶ Register

TAGS

Tags: Running only specific parts of a playbook instead of running the entire playbook.

Example:

- name: my play with tags
hosts: all
tasks:
- name: my task1 with tags
tags: my_first_tag
ping:
- name: my task2 with tags
tags: my_second_tag
ping:

```
ansible-playbook my-playbook.yml --tags my_first_tag
```

- ▶ Write simple playbook file
- ▶ Add two tasks (apt update – apt install nginx)
- ▶ Add tags to first task: update
- ▶ Add tags to second task: install
- ▶ Run only the (apt update) task
- ▶ Example: `ansible-playbook my-playbook.yml --tags update`
- ▶ Add one task with “tags: always” and run the previous command again



TAGS

VARIABLES

Variables: Ansible uses variables to manage differences between systems.

Locations:

- Playbooks
- Inventory
- Command Line
- Register
- Files or Roles

example:

```
- name: my play with variables
  hosts: all
```

vars:

```
  package: nginx
  version: latest
```

tasks:

```
- name: my task with variables
  apt:
    name: "{{ package }}"
    state: "{{ version }}"
```

Define these variables (package_name, package_version)

- ▶ on playbook level
- ▶ on inventory level
- ▶ on command line level

Use apt module with the package name and version from your variables

VARIABLES



LOOPS

Loops: Ansible uses loops to execute a task multiple times.

example:

```
- name: my play with loops
hosts: all
tasks:

- name: my task with loops
apt:
  name: "{{ item }}"
  state: latest
loop:
  - nginx
  - mariadb-server
  - curl
```

example:

```
- name: my play with loops
hosts: all
tasks:

- name: my task with loops
apt:
  name: "{{ item.package_name }}"
  state: "{{ item.package_state }}"
loop:
  - { package_name: "nginx", package_state: "present" }
  - { package_name: "mariadb-server", package_state: "latest" }
  - { package_name: "curl", package_state: "absent" }
```

- ▶ Loop over a list of packages and install latest versions.
- ▶ Loop over a list of packages and perform different actions as per input.



LOOPS

WHEN

When: You want to execute different tasks depending on the value of a fact, a variable, or the result of a previous task.

Example:

- name: my play with conditions

 - hosts: all

 - tasks:

- name: my task1 with conditions

 - apt:

 - name: nginx

 - when: ansible_facts['distribution'] == "Ubuntu"

- name: my task2 with conditions

 - apt:

 - name: httpd

 - when: ansible_facts['distribution'] == "CentOS"

- ▶ Install nginx or apache2 depending on distribution
- ▶ Restart nginx service if distribution is ubuntu and variable value is true

WHEN



REGISTER

Register: Ansible register is a way to capture the output from task execution and store it in a variable.

Example:

- name: my play with register
hosts: all
tasks:
 - name: my task1 with register
command: cat /var/www/html/index.html
register: my_result
 - name: my task2 with register
debug:
var: my_result

- ▶ View the value of your register variable using debug module
- ▶ Restart service if the installation task was changed or was not failed



REGISTER & WHEN